Tooling and environment

12B. Tooling & Environment

Questions: 5 **Difficulty:** Mid to Senior **Companies:** Google, Amazon, Flipkart, Razorpay, Microsoft, Atlassian

---

Q 12.7 What are monorepos? Compare Turborepo vs Nx.

Difficulty: Senior **Asked at:** Google, Amazon, Flipkart, Razorpay

A monorepo is a single repository containing multiple packages, applications, or services. Instead of separate repos for `frontend`, `design-system`, `utils`, and `mobile-app`, everything lives in one repo with shared tooling and dependencies.

Why monorepos:

- **Shared code without npm publishing:** Your design system component library is used by both the web app and the admin dashboard. In a polyrepo, you'd publish to npm and manage versions. In a monorepo, both apps import directly from the shared package — changes are immediate.
- **Atomic commits:** A breaking API change in the shared library + the updates in consuming apps all land in one commit. No version mismatch, no coordination between repos.
- **Consistent tooling:** One ESLint config, one TypeScript config, one CI pipeline for everything.

Workspace setup (all package managers support this):

```
// package.json (root)
{
  "workspaces": ["apps/*", "packages/*"]
}
```

JAVASCRIPT Copy

```
monorepo/
  ├── apps/
  │   ├── web/            ← Next.js frontend
  │   ├── admin/          ← Admin dashboard
  │   └── mobile/         ← React Native app
  ├── packages/
  │   ├── ui/             ← Shared component library
  │   ├── utils/          ← Shared utilities
  │   └── config/         ← Shared ESLint, TS configs
  ├── package.json
  └── turbo.json          ← (or nx.json)
```

Turborepo — task runner for monorepos:

Turborepo doesn't replace your package manager or bundler. It orchestrates tasks (build, test, lint) across packages with intelligent caching and parallelization.

JSON Copy

```
// turbo.json
{
  "tasks": {
    "build": {
      "dependsOn": ["^build"],    // Build dependencies first
      "outputs": ["dist/**"]       // Cache these outputs
    },
    "test": {
      "dependsOn": ["build"]
    },
    "lint": {}                     // No dependencies — run in parallel
```

BASH Copy

```
turbo run build
# First run: builds all packages in dependency order (~60 seconds)
# Second run: skips unchanged packages, uses cache (~2 seconds)
```

Turborepo's cache is the killer feature. If package `ui` hasn't changed since the last build, Turborepo restores the cached output instead of rebuilding. Remote caching (Vercel) shares this cache across the entire team — if your colleague already built `ui`, your machine downloads the cached result.

Nx — full monorepo framework:

Nx is more opinionated and feature-rich than Turborepo. It includes project generation (scaffolding), dependency graph visualization, affected-only test/build (only run tests for packages affected by the current change), and plugins for React, Angular, Next.js, and more.

BASH Copy

```
# Only test packages affected by the current changes
nx affected --target=test

# Visualize the dependency graph
nx graph
```

Turborepo vs Nx:

JAVASCRIPT Copy

```
Feature              | Turborepo          | Nx
  ———————————————————+————————————————————+————————————————————
  Philosophy         | Minimal task       | Full framework
                     | runner             | with opinions
  Learning curve     | Low                | Higher
  Caching            | Local + remote     | Local + remote
```

```
Affected analysis    | Basic                | Advanced (graph)
Caching              | No                   | Yes (generators)
Configuration        | turbo.json (small)   | nx.json + project.json
Best for             | Existing projects    | New large-scale projects
                     | adding caching       | from scratch
```

When monorepos don't work: When teams need completely independent deployment and technology choices (one team uses React, another uses Vue), or when the repo becomes so large that git operations slow down (thousands of developers, millions of files — at this scale, you need tools like Google's Bazel).

Follow-up: "Can you have a monorepo with different frameworks?"

Yes. `apps/web` can be Next.js, `apps/admin` can be Vite + React, `apps/landing` can be Astro. The shared `packages/ui` exports framework-agnostic components or provides framework-specific wrappers. This is common in organizations where different apps have different requirements but share a design system.

Q 12.8 What are source maps? How do they work and what are the security implications?

Difficulty: Mid **Asked at:** Google, Amazon, Flipkart

Source maps are files that map minified/bundled production code back to the original source. When an error occurs in production, the stack trace shows the original file names, line numbers, and variable names instead of `app.a7f3b2.js:1:42398`.

How they work:

The bundler generates a `.map` file alongside each output file:

JAVASCRIPT Copy

```
dist/
    app.a7f3b2.js       ← Minified bundle (served to users)
    app.a7f3b2.js.map   ← Source map (maps minified → original)
```

The minified file has a comment at the bottom pointing to the map:

JAVASCRIPT Copy

```
// ... minified code ...
//# sourceMappingURL=app.a7f3b2.js.map
```

The source map contains: original file paths, original line/column mappings, and optionally the original source code itself (depending on configuration).

In Chrome DevTools: If source maps are available, the Sources panel shows your original files with original variable names. Breakpoints, stepping through code, and error traces all reference your source, not the bundle.

Error monitoring (Sentry, Datadog): Upload source maps to your error monitoring service. When a production error occurs, the service uses the source map to translate the minified stack trace to the original code, showing you the exact line that failed.

BASH Copy

```
# Upload source maps to Sentry during build
npx sentry-cli sourcemaps upload --release=1.0.0 ./dist
```

Security implications:

Source maps contain your original source code. If they're publicly accessible, anyone can read your business logic, find vulnerabilities, and understand your application's internals.

Options:

- **Don't serve source maps to users.** Generate them, upload to Sentry/Datadog, but don't deploy the `.map` files to your CDN. The `//# sourceMappingURL` comment can be stripped or pointed to an authenticated endpoint.
- **Serve only to authenticated users.** Configure your server to return source maps only with proper authentication headers.

- **Hidden source maps.** Vite's `build.sourcemap: 'hidden'` generates source maps without the `//# sourceMappingURL` comment. The files exist for upload to error services, but browsers don't know about them.

JAVASCRIPT Copy

```
// vite.config.js
export default {
  build: {
    sourcemap: 'hidden',  // Generate maps but don't reference them in out
  },
};
```

Q 12.9 How do environment variables work in frontend projects? What's the security concern?

Difficulty: Mid **Asked at:** Razorpay, Flipkart, Swiggy, Amazon

Environment variables let you configure your app differently for development, staging, and production without changing code. But in frontend, they work fundamentally differently from backend.

The critical distinction:

Backend environment variables are truly secret — they exist only on the server, never sent to the client. Frontend "environment variables" are replaced at build time and baked into the JavaScript bundle. They're public.

BASH Copy

```
# .env
VITE_API_URL=https://api.example.com
VITE_STRIPE_PUBLIC_KEY=pk_live_abc123
DATABASE_URL=postgresql://user:pass@db.example.com  # Backend only!
```

JAVASCRIPT Copy

```
// In the production bundle, this becomes:
console.log("https://api.example.com");
// The original variable name is gone — it's a literal string in the bundl
```

The prefix convention:

JAVASCRIPT Copy

Framework	Prefix	Exposed to browser?
Vite	VITE_	Yes
Next.js	NEXT_PUBLIC_	Yes
CRA	REACT_APP_	Yes
(none)	(no prefix)	No — server only

Any variable with the prefix is embedded in the client bundle and visible to anyone viewing your JavaScript. Variables without the prefix are only available in server-side code (API routes, server components, build scripts).

What's safe to put in frontend env vars:

- Public API URLs
- Public keys (Stripe publishable key, Google Maps API key)
- Feature flags
- Analytics IDs
- App version numbers

What must NEVER be in frontend env vars:

- API secret keys
- Database credentials
- JWT signing secrets

JAVASCRIPT Copy

```
// DANGEROUS: This secret is baked into the client bundle
NEXT_PUBLIC_STRIPE_SECRET_KEY=sk_live_xyz789
// Anyone opening DevTools → Sources can read this

// SAFE: This stays on the server
STRIPE_SECRET_KEY=sk_live_xyz789
// Only accessible in API routes / server components
```

Build-time vs runtime variables:

Standard env vars are replaced at build time. If you need to change a value without rebuilding (switch API endpoint, toggle feature flag), you need runtime configuration:

JAVASCRIPT Copy

```
// Runtime config — loaded at app startup, not baked in at build time
async function loadConfig() {
  const response = await fetch('/config.json');
  return response.json();
}
// config.json is served by your server and can be changed without redeplo
```

Follow-up: "How do you handle different environments (dev, staging, production)?"

Use `.env.development`, `.env.staging`, `.env.production` files. Vite and Next.js load the right file based on the build mode. In CI, environment variables are set through the CI platform (GitHub Actions secrets, Vercel environment variables), not committed to the repo.

Q 12.10 What DevTools features should every frontend engineer master?

This isn't a "list the panels" question. Interviewers want to know you can use DevTools to solve real problems, not just inspect elements.

Performance tab — the most underused panel:

Record a page load or interaction, and the flame chart shows exactly what the browser did, millisecond by millisecond. What to look for:

- **Long Tasks** (red triangles): Any JavaScript execution >50ms. Click to see which function caused it.
- **Layout shifts**: Yellow blocks labeled "Layout" — hover to see which elements triggered reflow.
- **Network waterfall**: See which resources blocked rendering and which downloaded in parallel.
- **Frame drops**: The green bar at the top shows frames. Dips below 60fps indicate jank.

Memory tab — detecting leaks:

Take a heap snapshot before an action (open/close a modal 10 times), take another snapshot after, and compare. Growing "Detached HTMLElement" counts indicate DOM nodes that were removed from the page but still held in memory by JavaScript references. Growing event listener counts suggest unremoved listeners.

Network tab — beyond basic inspection:

- **Throttling**: Switch to "Slow 3G" to experience your app as users on slow networks do.
- **Block request URL**: Right-click a request → "Block request URL" — see what happens when a CDN fails or a script doesn't load.
- **Copy as fetch/cURL**: Right-click a request → "Copy as fetch" — instant reproducible API call for debugging.
- **Waterfall column**: Stacked bars show DNS, TCP, TLS, request, response timing for each resource. Tall green bars = slow server response. Tall blue bars = slow download.

Console — advanced usage:

```
// Structured logging with groups
console.group('Cart Update');
console.log('Items:', items);
console.table(items);  // Renders array of objects as a table
console.groupEnd();

// Monitor function calls without modifying code
monitor(functionName);  // Logs every call with arguments

// Track DOM element changes
monitorEvents(element, 'click');  // Logs all click events on the element

// Performance measurement
console.time('render');
renderExpensiveComponent();
console.timeEnd('render');  // "render: 127ms"
```

React DevTools Profiler:

Record a session, interact with the page, and see: which components rendered, why they rendered (state change, props change, parent render), and how long each render took. Sort by render time to find the bottleneck. Components highlighted in yellow/red are the slow ones.

Lighthouse tab:

Run performance, accessibility, best practices, and SEO audits. Each recommendation links to specific code fixes. Run in incognito (no extensions) for accurate results.

Follow-up: "Walk me through how you'd debug a slow page."

1. Performance tab → record page load → identify the longest tasks in the flame chart
2. Network tab → check waterfall for slow resources or blocking chains
3. Lighthouse → get a baseline score and specific recommendations
4. React Profiler → identify components with expensive renders
5. Memory tab → check for leaks if the page slows down over time

This systematic approach — measure first, then optimize the bottleneck — is what interviewers want to hear.

QUICK CHECK — no marks, just for you.

1. In a Next.js project, which of these .env entries stays server-only and is NOT baked into the JavaScript bundle shipped to the browser?

- NEXT_PUBLIC_API_URL=https://api.example.com
- STRIPE_SECRET_KEY=sk_live_xyz789
- NEXT_PUBLIC_FEATURE_FLAG=true
- NEXT_PUBLIC_STRIPE_SECRET_KEY=sk_live_xyz789

Check answer

2. You want Sentry to de-minify your production stack traces, but you do not want browsers or curious users to discover the source maps. What does Vite's build.sourcemap: 'hidden' setting do?

- It generates the .map files and makes the server return them only for authenticated requests.
- It skips generating source maps entirely, so no .map files are emitted at all.
- It generates the .map files but omits the //# sourceMappingURL comment from the bundle output.
- It encrypts the .map files so that only the error monitoring service can decode them.

Check answer

3. A team is starting a new large-scale monorepo from scratch and wants both scaffolding generators and the ability to run tests only for packages affected by the current change.

Which statement about Turborepo and Nx is accurate?

- Nx offers code generators and advanced graph-based affected analysis, while Turborepo has no code generation.
- Neither tool can tell which packages a change affects, so every task must be run for every package.
- Only Nx supports remote caching; Turborepo's cache is strictly local to each machine.
- Turborepo offers code generators, while Nx is limited to task caching and parallelization.

Check answer

Previous
Modules and bundlers

Next
Code Management

LEARN MORE

ARTICLES & DOCS

- ESLint
  eslint.org
- Prettier
  prettier.io

VIDEOS



Tooling: Webpack, Babel, ESLint, Prettier
YouTube



ESLint + Prettier + VS Code setup
YouTube



Frontend System Desi...

Hand-picked references for this lesson.